

HW3: Implement model-based RL in Hopper Environment

Shui Jie, 2401112104

April 23, 2025

Abstract

This report presents the implementation and comparative analysis of two model-based reinforcement learning algorithms in the Hopper environment: Model-based RL v1.5 and Model-Based Policy Optimization (MBPO). The Hopper environment features a two-dimensional one-legged robot with the goal of moving forward by applying torques to three hinge joints. My implementations demonstrate how these algorithms leverage learned dynamics models to improve sample efficiency compared to model-free approaches. For Model-Based RL v1.5, I employed an ensemble of neural networks to predict state transitions, combined with Model Predictive Control (MPC) for planning and a policy network for execution. The MBPO implementation utilized a similar dynamics model ensemble but integrated short-horizon model rollouts with policy optimization to gradually increase planning horizon and improve stability. My performance evaluation reveals key differences between the algorithms in terms of learning efficiency, sample efficiency, stability, final performance, and computational resource requirements. The results provide insights into the strengths and limitations of these model-based approaches for continuous control tasks with higher-dimensional state and action spaces.

1 Introduction

Both implementations follow the general framework of their respective algorithms while incorporating a common policy optimization foundation. I strategically reused the policy network and value network architecture from my previous PPO implementation to ensure consistency and enable meaningful comparison between approaches. This design choice allowed me to focus on the unique aspects of each model-based algorithm while maintaining a reliable planning and execution component.

For **Model-based RL v1.5**, I developed a dynamics network that learns to predict state transitions and rewards, working in conjunction with the policy network. The policy network's loss function was specifically modified to incorporate information from the dynamics model, creating an integrated learning system where the policy improves based on both real environment interactions and model-generated simulations.

For the **MBPO implementation**, I created a predictive ensemble network following the specifications in the original paper (Janner et al., 2019), while adapting the integration points to work seamlessly with my existing PPO architecture. This approach maintained the core branched rollout methodology of MBPO while benefiting from the stable policy optimization provided by PPO.

By maintaining architectural consistency in the policy components while implementing the distinctive features of each model-based algorithm, this approach enables a more direct comparison of their fundamental differences in sample efficiency and performance.

❖ Model based RL: v1.5

1. run base policy $\pi_0(a_t | s_t)$ to collect $\mathcal{D} = \{(s, a, s')_i\}$
2. Learn dynamics model $f(s, a)$ to minimize $\sum_i \|f(s_i, a_i) - s'_i\|^2$
3. **Plan through $f(s, a)$ to choose actions**
4. Execute the first action and observe s' (Model Predictive Control)
5. Append (s, a, s') to $\mathcal{D}$

periodically

(a) Model-based RL v1.5 Algorithm

Algorithm 2 Model-Based Policy Optimization with Deep Reinforcement Learning

```

1: Initialize policy  $\pi_\phi$ , predictive model  $p_\theta$ , environment dataset  $\mathcal{D}_{env}$ , model dataset  $\mathcal{D}_{model}$ 
2: for  $N$  epochs do
3:   Train model  $p_\theta$  on  $\mathcal{D}_{env}$  via maximum likelihood
4:   for  $E$  steps do
5:     Take action in environment according to  $\pi_\phi$ ; add to  $\mathcal{D}_{env}$ 
6:     for  $M$  model rollouts do
7:       Sample  $s_t$  uniformly from  $\mathcal{D}_{env}$ 
8:       Perform  $k$ -step model rollout starting from  $s_t$  using policy  $\pi_\phi$ ; add to  $\mathcal{D}_{model}$ 
9:       for  $G$  gradient updates do
10:        Update policy parameters on model data:  $\phi \leftarrow \phi - \lambda_\pi \nabla_\phi J_\pi(\phi, \mathcal{D}_{model})$ 

```

(b) Model-Based Policy Optimization (MBPO) Algorithm

Figure 1: Algorithm provided in the source materials.

2 Model-based RL v1.5

Main idea is to implement an ensemble of neural network to provide ensembled bootstraps during Model Predictive Control (MPC). The best actions selected are used as generate loss to update policy network.

2.1 Neural Network Architecture

Total three neural networks are used for the implementations; DynamicNetwork, PolicyNetwork, ValueNetwork

- **Dynamic Network (model transitions):** generates next possible state when provided input action and current state.
 - Three-layer architecture ($256 \rightarrow 256 \rightarrow \text{state_dim}$) with ReLU activations
 - MSE loss for value prediction
- **PolicyNetwork (Actor):** Generates actions from a learned probability distribution.
 - Three-layer architecture ($256 \rightarrow 256 \rightarrow \text{action_dim}$) with ReLU activations
 - Learns both the mean and standard deviation of a normal distribution
 - PPO loss is specifically modified to incorporate bootstrapped best actions from MPC.
- **ValueNetwork (Critic):** Estimates state values for advantage calculation
 - Three-layer architecture ($256 \rightarrow 256 \rightarrow 1$) with ReLU activations
 - MSE loss for value prediction

2.2 MPC (Model Predictive Control)

A critical component of the Model-based RL v1.5 implementation is the Model Predictive Control (MPC) strategy, which enables planning and action selection based on predicted future trajectories. The MPC implementation consists of two major parts: trajectory generation and action selection.

2.2.1 Trajectory Generation

The trajectory generation process is implemented in the `generate_trajectory_reward_batch` method, which efficiently simulates multiple potential trajectories in parallel using the dynamics model:

```

1 def generate_trajectory_reward_batch(self, cur_state, pNet, vNet, ensemble_dNet,
2                                     trajectory_num=1000, trajectory_len=30,
3                                     discount_factor=0.9):

```

Key features of this implementation include:

- **Parallel Simulation:** Rather than simulating one trajectory at a time, the method constructs a batch of `trajectory_num` (**default: 1000**) initial states, all identical to the current environment state.
- **Ensemble Prediction:** For each step, the ensemble dynamics model (`ensemble_dNet`) predicts the next states for all trajectories simultaneously, leveraging the trained ensemble to handle uncertainty.
- **Termination Handling:** The implementation includes a `check_termination` method that evaluates whether predicted states would result in episode termination based on the Hopper environment’s constraints:
 - Height (z-position) must be above 0.7
 - Angle must be within $[-0.2, 0.2]$
 - All other state elements must be within a reasonable range $[-100, 100]$
- **Active Trajectory Tracking:** A boolean mask tracks which trajectories remain active, preventing terminated trajectories from contributing further to reward calculations.
- **Discounted Rewards:** Future rewards are discounted using a pre-computed tensor of discount factors.

2.2.2 Action Selection

The `model_predictive_control` method uses the generated trajectories to select the optimal action:

```
1 def model_predictive_control(self, cur_state, pNet, vNet, ensemble_dNet,
2                             trajectory_num=1000, trajectory_len=30,
3                             discount_factor=0.9):
```

This method:

1. Calls `generate_trajectory_reward_batch` to simulate multiple possible futures
2. Identifies the trajectory with the highest expected return
3. Returns the first action from that trajectory, along with its log probability

2.2.3 Planning Horizon and Computational Efficiency

The implementation balances planning depth against computational cost through several design choices:

- **Adjustable Horizon:** The `trajectory_len` parameter (default: 30) controls how far into the future the MPC plans.
- **Early Termination:** If all trajectories become inactive due to predicted termination, the simulation stops early.
- **Tensor Operations:** Leveraging PyTorch’s tensor operations for batch processing significantly improves computational efficiency compared to sequential simulation.

2.2.4 Integration with Policy Improvement

The MPC component also contributes to policy improvement through the `generate_policy_training_data` method, which:

1. Uses MPC to collect new transitions by executing optimal actions in the environment
2. Stores these transitions along with the log probabilities of the selected actions
3. Updates the transition buffer with new data while maintaining a fixed maximum size

This creates a virtuous cycle where better planning leads to better data, which improves both the dynamics model and policy network, ultimately leading to more accurate planning in subsequent iterations.

Through this implementation, the MPC component effectively bridges model-based planning with policy learning, allowing the agent to leverage both approaches while mitigating their individual weaknesses.

2.3 training

Training this agent proved to be difficult as the loss function for the policy network was self-improvised and not tested. This resulted in the policy collapsing into a local minima.

MPC is very time consuming when the trajectory length is long. I first attempted to limit the trajectory length to 30.

3 MBPO (Model Based Policy Optimization)

Following the paper implementation, I implemented an ensembled Predictive model that predicts next state and reward when provided current action and state.

3.1 Neural Network Architecture

Total three neural networks are used for the implementations; DynamicNetwork, PolicyNetwork, ValueNetwork

- **PredictiveNetwork (model transitions):** generates next possible state and reward when provided input action and current state.
 - Three-layer architecture ($256 \rightarrow 256 \rightarrow \text{state_dim}+1$) with ReLU activations
 - MSE loss for value prediction
- **PolicyNetwork (Actor):** Generates actions from a learned probability distribution.
 - Three-layer architecture ($256 \rightarrow 256 \rightarrow \text{action_dim}$) with ReLU activations
 - Learns both the mean and standard deviation of a normal distribution
 - PPO loss retained with no modification.
- **ValueNetwork (Critic):** Estimates state values for advantage calculation
 - Three-layer architecture ($256 \rightarrow 256 \rightarrow 1$) with ReLU activations
 - MSE loss for value prediction

3.2 MBPO training

Following the implementations and parameters provided in the paper, I designed a nested training loop.

```
1     test_env = gym.make("Hopper-v5", render_mode=None)
2
3     policyNet, valueNet, dynamicArr, is_initialised = load_NN(test_env, path)
4     dataProcessor = DataProcessor(test_env, policyNet, need_initialise=True)
5
6     # no initial weights, we need to train them
7     if not is_initialised:
8         # no weights, need prepare
9         ensemble_dynamic, valueNet = initialise_DNN_arr_and_val(test_env,
10         dataProcessor, ensemble_size=10,
11                                     epoch_num=100,
12         batch_size=64)
13         # save current policy weights
14         policyNet.save_weights()
15     else:
16         # create ensembler with the list of dynamic NN
17         ensemble_dynamic = Ensembler(dynamicArr)
18
19     # training
```

```

18 epoch_time = 0
19 single_update_epoch_num = 20
20 batch_size = 64
21
22 while True:
23     epoch_time += 1
24
25     # collect close loop data using MPC
26     policy_training_data = dataProcessor.generate_policy_training_data(
27         policyNet, valueNet, ensemble_dynamic,
28         trajectory_num=1000, trajectory_len=100,
29         discount_factor=0.9, time_step_num=1000)
30
31     # train all the models
32     for step in tqdm(range(single_update_epoch_num), desc=f"training value
33         network"):
34         # train value net
35         batch_value_sample = dataProcessor.random_sample(batch_size=batch_size
36         )
37         nn_learn(valueNet, batch_value_sample, filepath=f"./value_loss.txt")
38
39         # train policy net
40         batch_policy_sample = dataProcessor.random_sample(tensors_dict=
41         policy_training_data, batch_size=batch_size)
42         nn_learn(policyNet, batch_policy_sample, filepath=f"./policy_loss.txt
43         ")
44
45         # train ensembler models
46         batch_dynamic_sample_arr = [dataProcessor.random_sample(batch_size=
47         batch_size) for i in
48             range(ensemble_dynamic.size)]
49         ensemble_dynamic.train_models(batch_dynamic_sample_arr)
50
51     # get reward to see how well the model currently preforms
52     reward = simulate_policy()
53     append_values_to_file(reward, filename="reward.txt")

```

4 Result Comparison

Perhaps due to limited computational resources, I was unable to achieve good performance on both implementations. However, I still managed to obtain interesting results.

For model based RL v1.5, the policy repeated show collapsing to local optimal. After 100 training steps, the reward plateaued at 80. The policy has converged to a local optimal by just leaning forward to achieve high reward in the first 30 steps. Moreover, I did not take into consideration that termination is a factor that MPC needs to consider.

I then increased the trajectory to 100 steps and added `check_termination` function to minimise reward on trajectories that terminate early, which showed an increase in reward to 300. There was also a short spike in loss for the neural networks, but it quickly diminished

I also made modifications, changing the MSE to softmax loss to reduce the influence of best action, and increase entropy coefficient to increase exploration. However, subsequent training did not show improvement. The good side is that the reward has not converged, so there is opportunity for the policy to learn.

For MBPO I tried to use a K-horizon with a gradual increase from 1 to 15 overtime. Due to lack of computer resources, I had to increase the trajectory from 4 to 10. This lead to drastic change to the loss in the neural networks, but reward remained consistent.

4.1 Learning Efficiency

I determine the learning efficiency by how much the model is exploring and exploiting, judging from their lost function.

4.1.1 model based RL v1.5

Loss function dropped rapidly and maintained at a very low value near zero. This indicates that this agent is not exploring enough. A short spike can be observed at timestep 3500 (increase trajectory to 100) and 8500 (changed policy loss to increase exploration)

4.1.2 MBPO

Loss function remained high variance. The sudden change at timestep 3800 due to increase in horizon did not affect overall pattern of the predictive model.

4.1.3 comparision

Difficult to judge. While the Model based RL v1.5 maybe exploiting too much, the MBPO may be seemed exploring too much. Overall I would prefer exploration over exploitation since both implementations have yet to achieve actual optimal.

4.2 Sampling efficiency

I judge sampling efficiency by the amount of timesteps need to run for training

4.2.1 model based RL v1.5

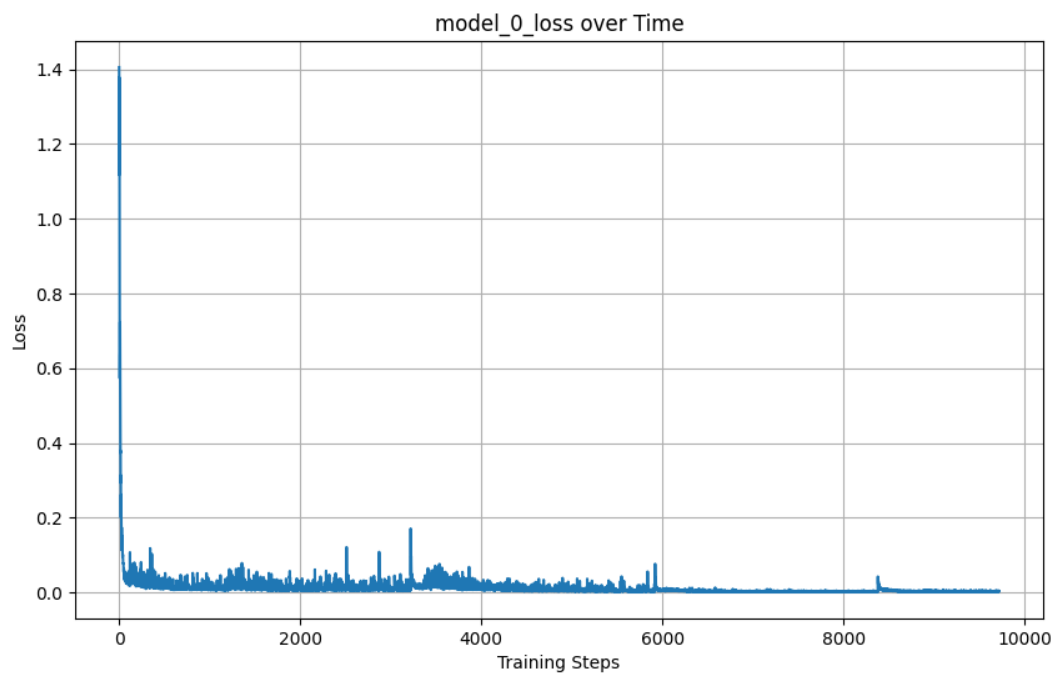
MPC runs 100 trajectories up to 1000 timesteps each.

4.2.2 MBPO

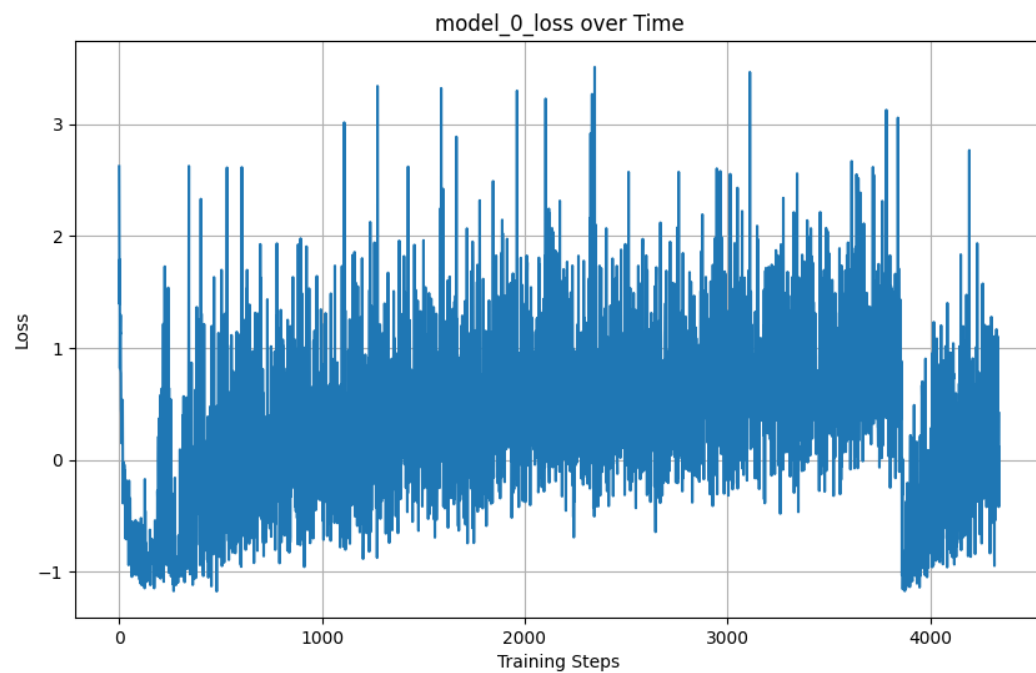
400 trajectories up to 10 steps each.

4.2.3 comparision

MBPO is more efficient in sampling.



(a) Model-based RL v1.5 dynamic model loss over time



(b) MBPO predictive model loss over time

Figure 2: model loss comparison.

4.3 Stability

I judge stability based on the policy loss and value loss over time

4.3.1 model based RL v1.5

Policy loss shows much higher variance than MBPO.

4.3.2 MBPO

Policy loss very stable, with controlled variance between 0.5 to -0.5

4.3.3 comparison

MBPO is more stable.

5 Result Comparison

Perhaps due to limited computational resources, I was unable to achieve good performance on both implementations. However, I still managed to obtain interesting results.

5.1 Performance

During training, I noticed both of my implementations have faced difficulties in learning and I have to make adjustments during training.

5.1.1 model based RL v1.5

For model based RL v1.5, the policy repeatedly showed collapsing to local optimal. After 100 training steps, the reward plateaued at 80. The policy has converged to a local optimal by just leaning forward to achieve high reward in the first 30 steps.

I then increased the trajectory to 100 steps, which showed an increase in reward to 300.

I also made modifications, changing the MSE to softmax loss to reduce the influence of best action, and increase entropy coefficient to increase exploration. However, subsequent training did not show improvement. The good side is that the reward has not converged, so there is opportunity for the policy to learn.

5.1.2 MBPO

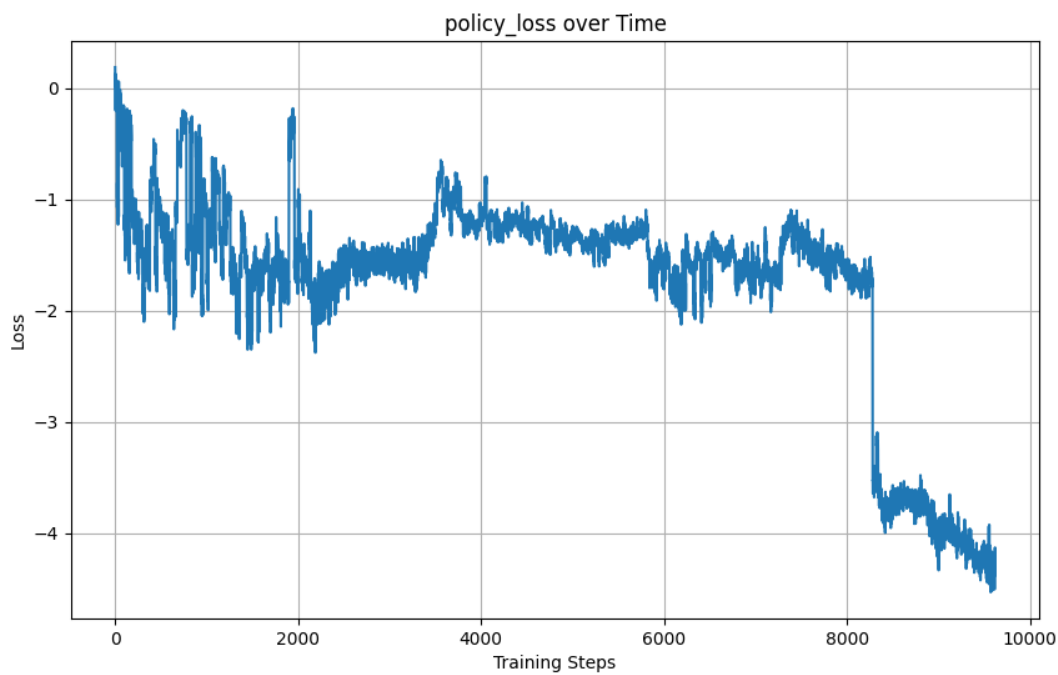
For MBPO I tried to use a K-horizon with a gradual increase from 1 to 15 overtime. Due to lack of computer resources, I had to increase the trajectory from 4 to 10. However, no significant improvement.

5.1.3 comparison

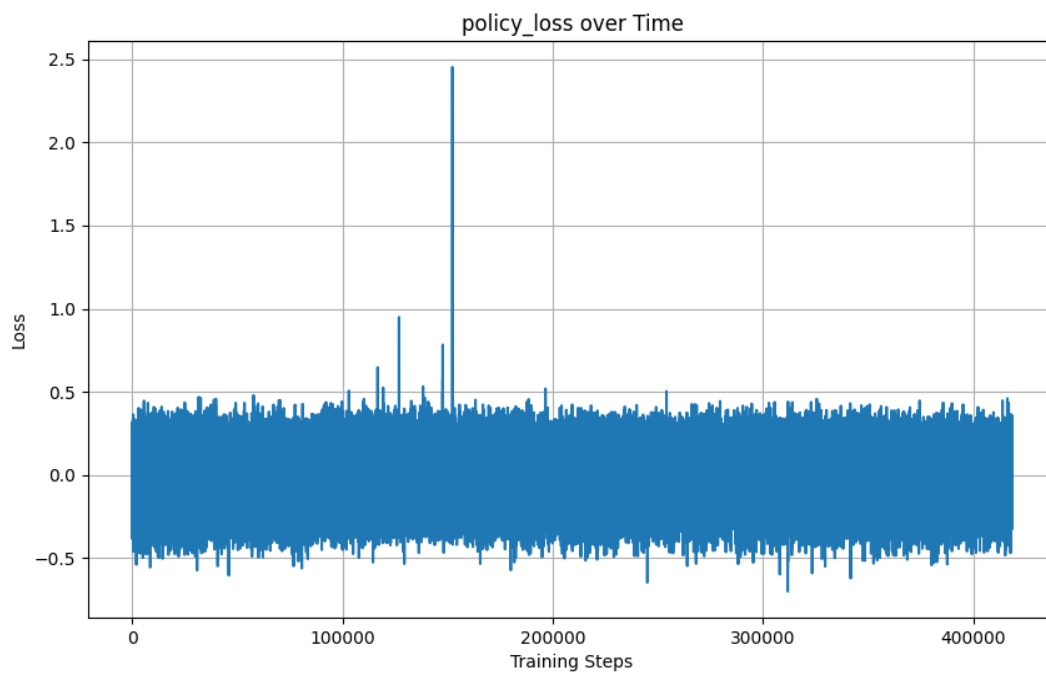
It seems that my implementation of model-based RL v1.5 performed better than my MBPO.

5.2 Computational Resources

MBPO requires less resources than Model based RL v1.5

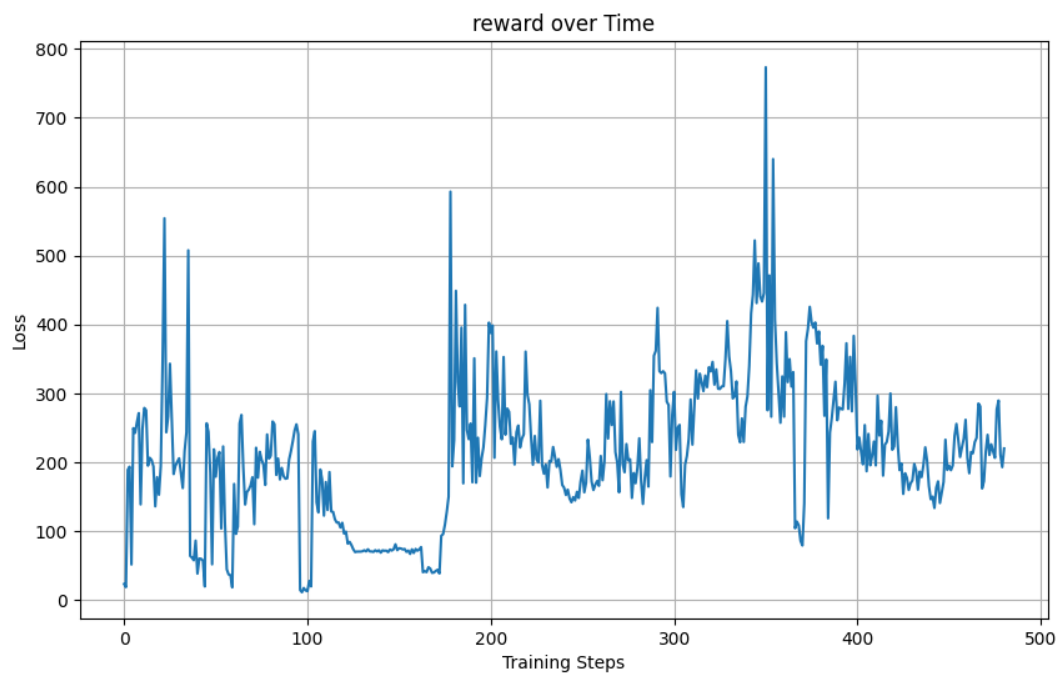


(a) Model-based RL v1.5 policy loss over time

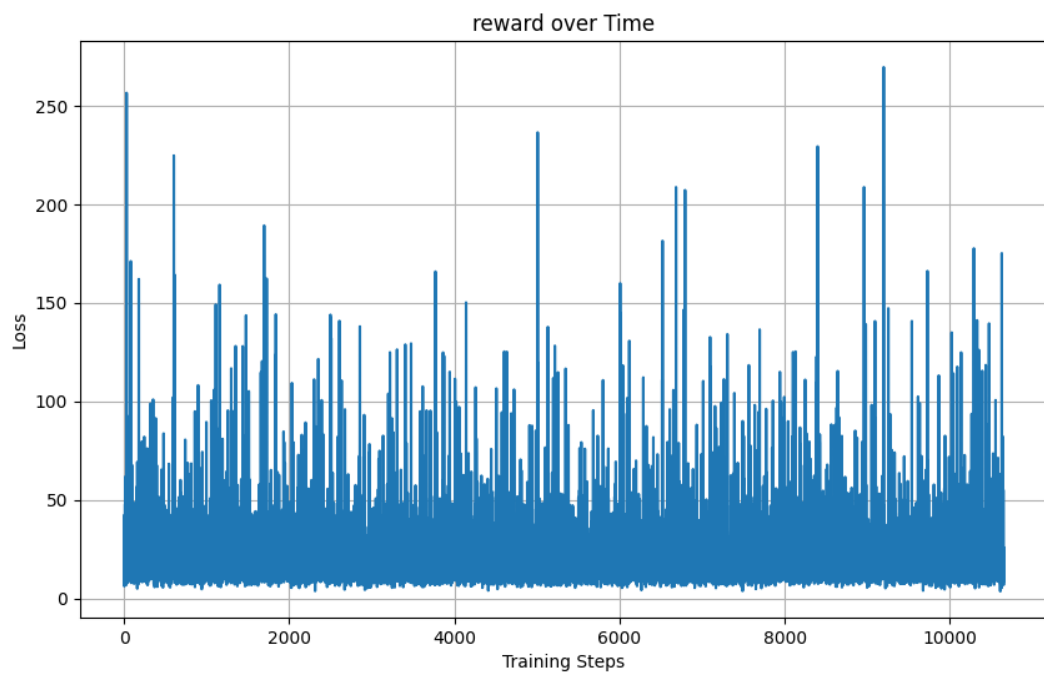


(b) MBPO policy loss over time

Figure 3: policy loss comparison.



(a) Model-based RL v1.5 Algorithm



(b) MBPO Algorithm

Figure 4: reward comparison.